MTM4692

Spring 2026

Fettah KIRAN

Department of AI and Data Engineering

fkiran@yildiz.edu.tr

Backup, Restore & Replication

High Availability — Disaster Recovery & Data Protection

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:05	Recap Quiz	Review
1:05 – 1:25	Backup Strategies Overview	**Theory** 📖
1:25 – 2:10	MySQL Backup Tools	**Theory** 📖
2:10 – 2:25	☕ Break (15 min)	
2:25 – 2:45	Replication Concepts	**Theory** 📖
2:45 – 3:00	RPO and RTO	**Theory** 📖
3:00 – 3:15	High Availability	**Theory** 📖
3:15 – 3:30	SQLite Backup Methods	**Practice** 💻
3:30 – 3:40	Practice & Homework	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Learning Objectives

By the end of this week you will be able to:

1. Compare **logical vs physical** backup strategies
2. Use **mysqldump** for logical backups
3. Understand **binary log** concepts
4. Explain **replication** types and architectures
5. Calculate **RPO** and **RTO** for disaster recovery
6. Design **high availability** solutions
7. Perform **SQLite backups** using built-in methods

Part 1: Backup Strategies Overview

Why Backups Matter

Data loss scenarios:

- 🔧 Hardware failure (disk crash)
- 👤 Human error (`DROP TABLE students;`)
- 🦠 Security breach (ransomware)
- 🌊 Natural disasters
- 🐛 Software bugs corrupting data

> **No backup = No recovery**
>
> "It's not IF you'll need a backup, it's WHEN"

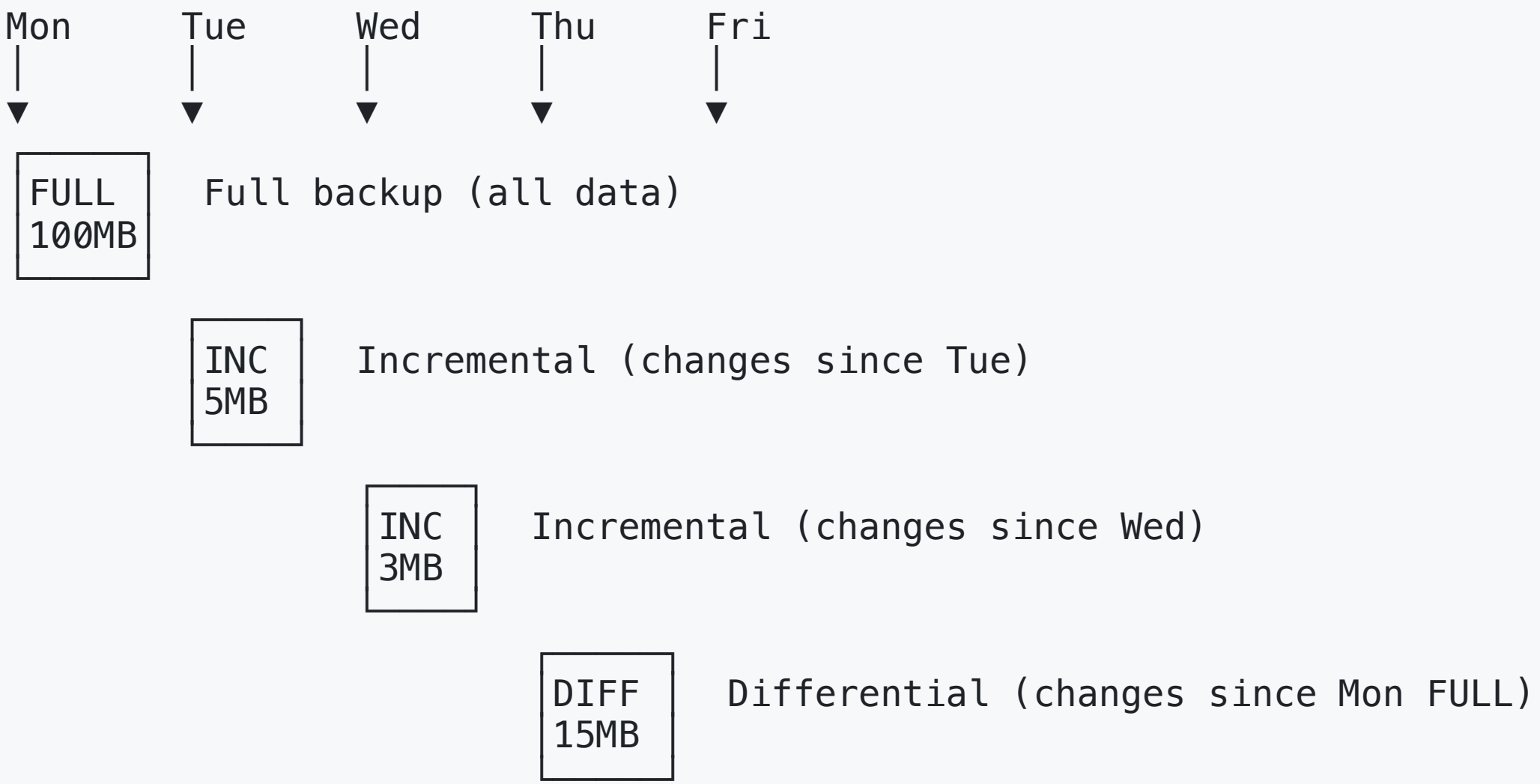
Backup Types

Type	Description	Method
Logical	SQL statements to recreate data	mysqldump
Physical	Copy of raw database files	File copy, Percona XtraBackup
Full	Complete copy of all data	Both methods
Incremental	Only changes since last backup	Binary logs
Differential	Changes since last full backup	Binary logs

Logical vs Physical Backups

Feature	Logical	Physical
Format	SQL text files	Binary files
Speed (backup)	Slower	Faster
Speed (restore)	Slower	Faster
Portability	✅ Cross-version, cross-platform	❌ Same version/platform
Granularity	Tables, databases	Entire server
Size	Smaller (compressed)	Larger
Tool	mysqldump	XtraBackup, file copy

Full vs Incremental vs Differential

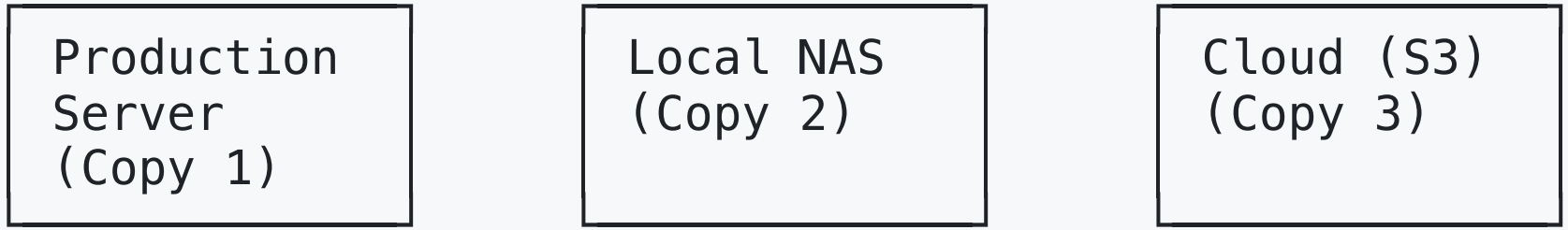


Backup Strategy: 3-2-1 Rule

3 copies of your data

2 different storage media

1 offsite (cloud, remote location)



Part 2: MySQL Backup Tools

mysqldump — The Standard Tool

```
-- Backup a single database
mysqldump -u root -p university > university_backup.sql

-- Backup specific tables
mysqldump -u root -p university student course > tables_backup.sql

-- Backup all databases
mysqldump -u root -p --all-databases > full_backup.sql

-- Backup with structure only (no data)
mysqldump -u root -p --no-data university > schema_only.sql

-- Backup with data only (no structure)
mysqldump -u root -p --no-create-info university > data_only.sql
```

mysqldump Output Format

```
-- mysqldump output is plain SQL:

-- Table structure
DROP TABLE IF EXISTS `student`;
CREATE TABLE `student` (
    `student_id` int NOT NULL AUTO_INCREMENT,
    `first_name` varchar(50) NOT NULL,
    `last_name` varchar(50) NOT NULL,
    `gpa` decimal(3,2) DEFAULT NULL,
    PRIMARY KEY (`student_id`)
) ENGINE=InnoDB;

-- Data
INSERT INTO `student` VALUES
    (1, 'Alice', 'Johnson', 3.95),
    (2, 'Bob', 'Smith', 3.80),
    (3, 'Carol', 'Davis', 3.20);
```

mysqldump Important Options

Option	Description
`--single-transaction`	Consistent backup for InnoDB
`--routines`	Include stored procedures & functions
`--triggers`	Include triggers (default: yes)
`--events`	Include scheduled events
`--add-drop-table`	Include DROP TABLE before CREATE
`--compress`	Compress client-server traffic
`--lock-tables`	Lock tables during backup
`--master-data`	Include binary log position

Best Practice mysqldump Command

```
# Production-ready backup command
mysqldump -u backup_user -p \
    --single-transaction \
    --routines \
    --triggers \
    --events \
    --add-drop-table \
    --set-gtid-purged=OFF \
    university > university_$(date +%Y%m%d_%H%M%S).sql

# Compressed backup
mysqldump -u root -p university | gzip > backup_$(date +%F).sql.gz
```

Binary Logs

Binary logs record **every change** made to the database:

```
# Enable binary logging (in my.cnf)
[mysqld]
log-bin = mysql-bin
binlog_format = ROW
expire_logs_days = 7
```

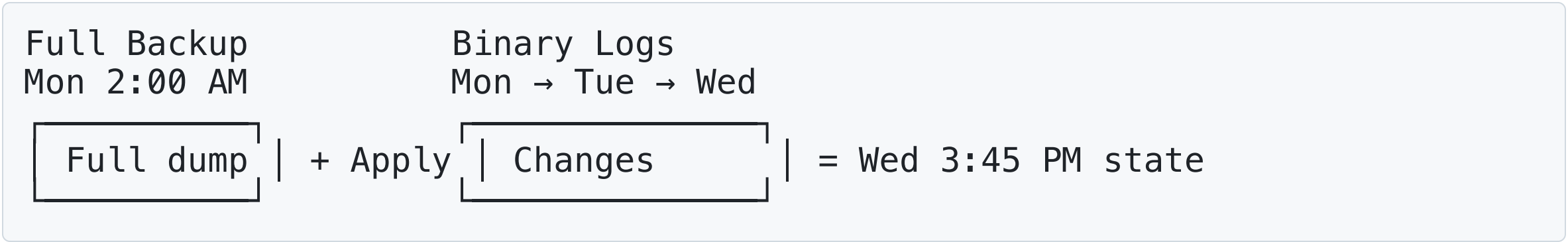
```
# List binary log files
SHOW BINARY LOGS;

# View binary log contents
mysqlbinlog mysql-bin.000001

# View events in readable format
SHOW BINLOG EVENTS IN 'mysql-bin.000001';
```

Point-in-Time Recovery (PTIR)

Binary logs enable recovering to a **specific moment**:




```
# Restore full backup
mysql -u root -p university < monday_backup.sql

# Apply binary logs up to a specific time
mysqlbinlog --stop-datetime="2026-01-15 15:45:00" \
    mysql-bin.000042 | mysql -u root -p university
```

🧠 Exam Focus — Backup Tools

mysqldump → Logical backup (SQL text)

Binary logs → Record all changes for incremental backup

--single-transaction → Consistent InnoDB backup without locking

Point-in-time recovery → Full backup + binary logs

mysqlbinlog → Tool to read binary log files

Part 3: Restore Operations

Restoring from mysqldump

```
# Restore a database
mysql -u root -p university < university_backup.sql
mysql -u username -p university < university_backup.sql

# Restore from compressed backup
gunzip < backup_2025-01-15.sql.gz | mysql -u root -p university

# Restore specific tables (if backup has them)
mysql -u root -p university < tables_backup.sql

# Restore to a new database
mysql -u root -p -e "CREATE DATABASE university_restored"
mysql -u root -p university_restored < university_backup.sql
```

Full Recovery Procedure

```
Step 1: Stop the application

Step 2: Restore the last full backup
        mysql -u root -p < full_backup.sql

Step 3: Apply incremental backups (binary logs)
        mysqlbinlog mysql-bin.000042 | mysql -u root -p

Step 4: Verify data integrity
        CHECK TABLE student;

Step 5: Restart the application
        Bring your app servers back online.
```

Verifying a Backup

```sql
-- After restore, verify:

-- 1. Table count
SELECT COUNT(*) AS tables
FROM information_schema.tables
WHERE table_schema = 'university';

-- 2. Row counts
SELECT table_name,
       table_rows AS approx_rows
FROM information_schema.tables
WHERE table_schema = 'university';

-- 3. Checksums
CHECKSUM TABLE student, course, enrollment;

-- 4. Check for corruption
CHECK TABLE student EXTENDED;
```

What is the issue here?

- **Extreme Downtime:** Restoring from a .sql file (logical backup) is single-threaded and painfully slow for large datasets.
- **Point of Failure:** Single-threaded binary log replay creates a massive replication lag and delays recovery.
- **Total Disruption:** Step 1 requires stopping the application completely, which violates high-availability (HA) requirements.

Part 4: Replication Concepts

What is Replication?

Replication = automatically copying data from one server to others

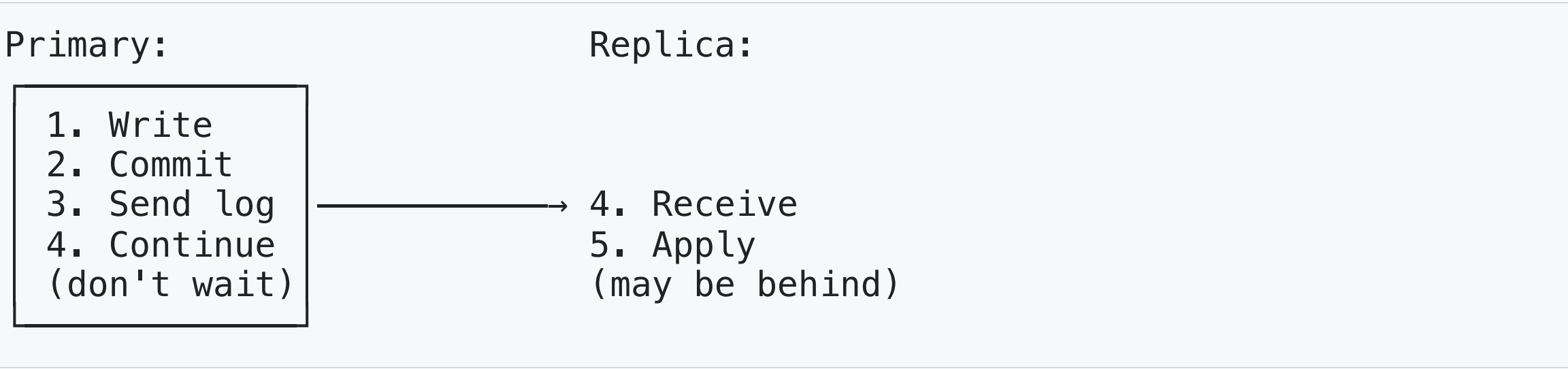
Why replicate?

- **Read scaling** — distribute read queries
- **High availability** — failover if primary fails
- **Disaster recovery** — copy in another location
- **Analytics** — run reports on replica

Replication Types

Type	Description	Lag
Asynchronous	Primary doesn't wait for replica	May have lag
Semi-synchronous	Primary waits for at least one replica ACK	Minimal lag
Synchronous	Primary waits for ALL replicas	No lag, slower
Group Replication	Multi-primary, consensus-based	Minimal lag

Asynchronous Replication



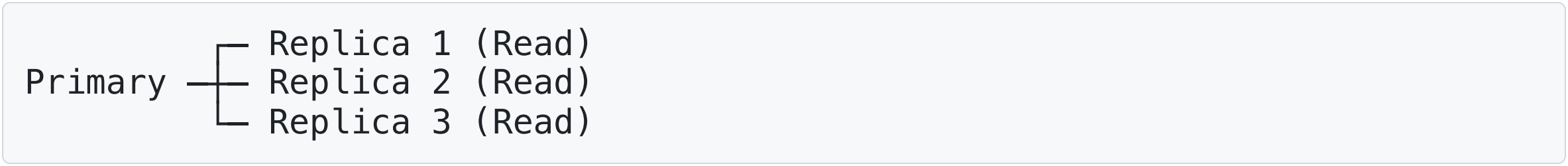
- **Default** in MySQL
- Primary doesn't wait → **fastest** for writes
- Replica may be **seconds behind** (replication lag)

Semi-Synchronous Replication

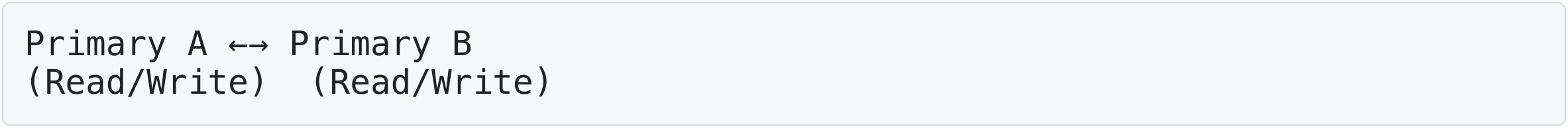
- Primary waits for **at least one** replica to receive the log
- Better **durability** — data exists on 2+ servers before commit returns
- Slightly slower than async

Replication Topologies

Single Primary (Master-Slave)



Multi-Primary (Master-Master)



Chain Replication

Setting Up Basic Replication (Theory)

```
-- On PRIMARY:
-- 1. Enable binary logging (my.cnf)
[mysqld]
server-id = 1
log-bin = mysql-bin

-- 2. Create replication user
CREATE USER 'replicator'@'%' IDENTIFIED BY 'secure_pass';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'%';

-- 3. Get binary log position
SHOW MASTER STATUS;
```

```
-- On REPLICA:
-- 4. Configure replica
CHANGE MASTER TO
    MASTER_HOST = '192.168.1.100',
    MASTER_USER = 'replicator',
    MASTER_PASSWORD = 'secure_pass',
```

🧠 Exam Focus — Replication

Asynchronous — Default, fastest, may lose committed data on crash

Semi-synchronous — Waits for one ACK, better durability

Synchronous — Waits for all, slowest, strongest guarantee

Primary handles **reads + writes**, replicas handle **reads only**

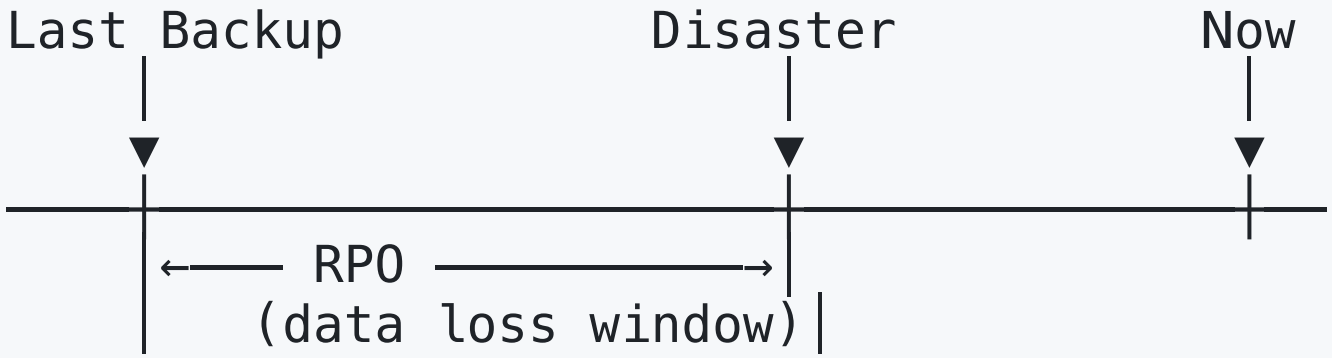
Replication uses **binary logs** to send changes to replicas

Part 5: RPO and RTO

- RPO — Recovery Point Objective
- RTO — Recovery Time Objective

RPO — Recovery Point Objective

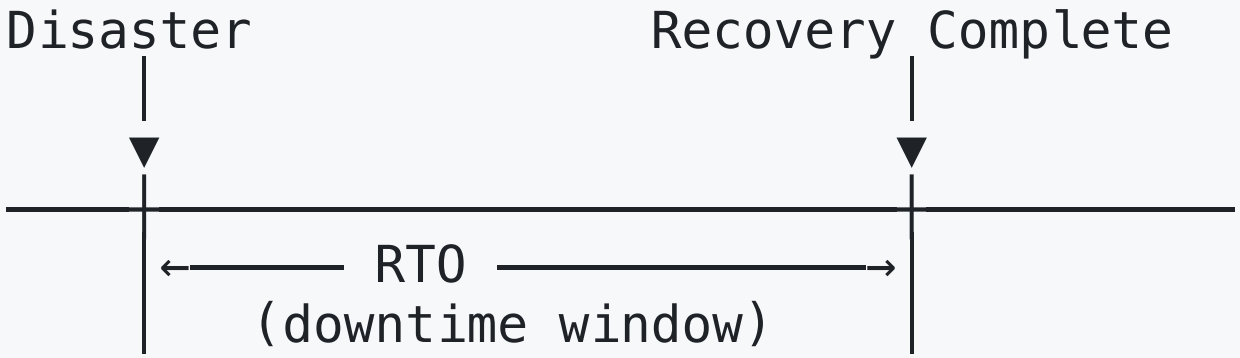
How much data can you afford to lose?



RPO	Meaning	Strategy
24 hours	Lose up to 1 day	Daily full backups
1 hour	Lose up to 1 hour	Hourly incremental
0	No data loss	Synchronous replication

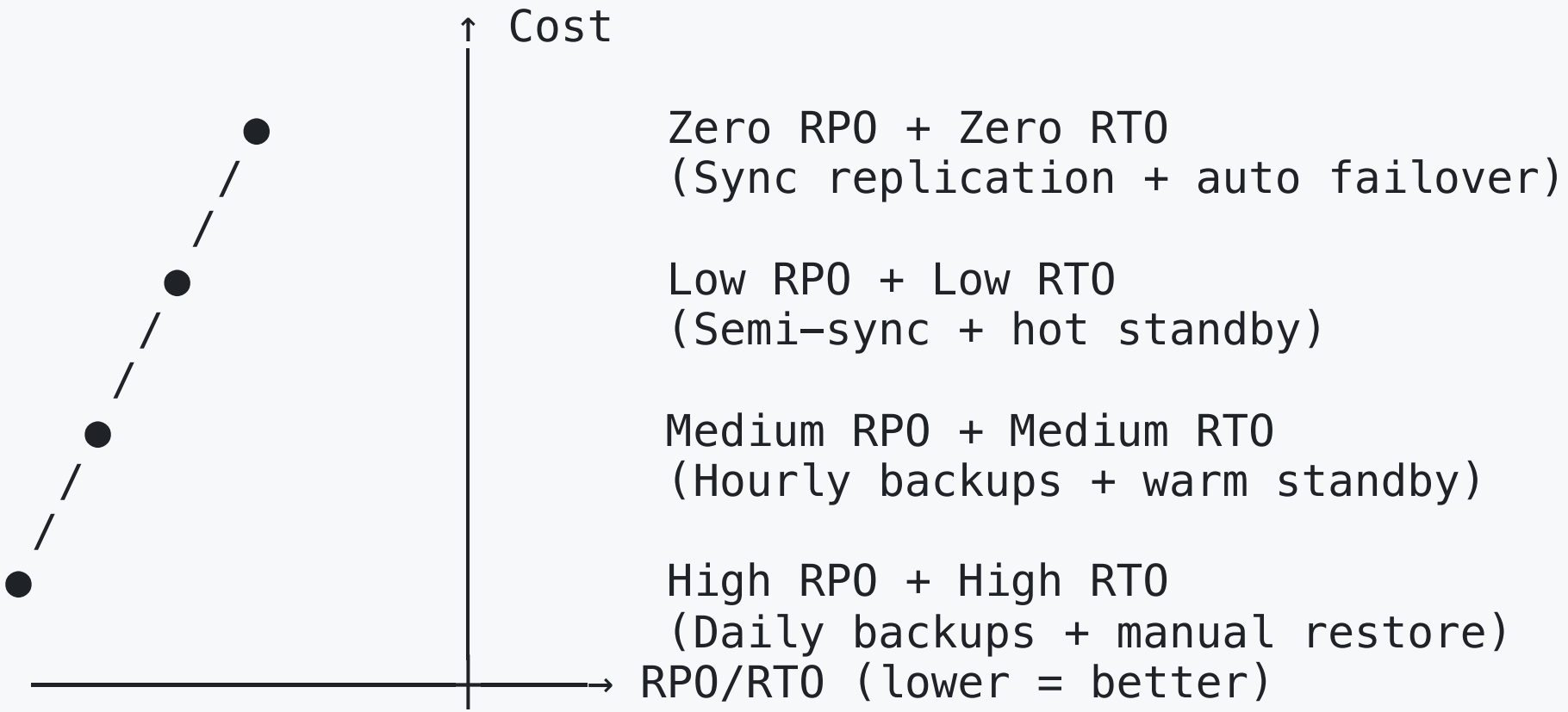
RTO — Recovery Time Objective

How long can you afford to be down?



RTO	Meaning	Strategy
24 hours	Can be down for 1 day	Restore from backup
1 hour	Down for max 1 hour	Hot standby
Minutes	Near-zero downtime	Auto-failover cluster

RPO vs RTO Trade-offs



Lower RPO/RTO = Higher cost

🧠 Exam Focus — RPO and RTO

RPO = How much data loss is acceptable

RTO = How much downtime is acceptable

RPO = 0 → **No data loss** → Synchronous replication needed

RTO = 0 → **No downtime** → Auto-failover cluster needed

Lower RPO/RTO requires **more expensive** infrastructure

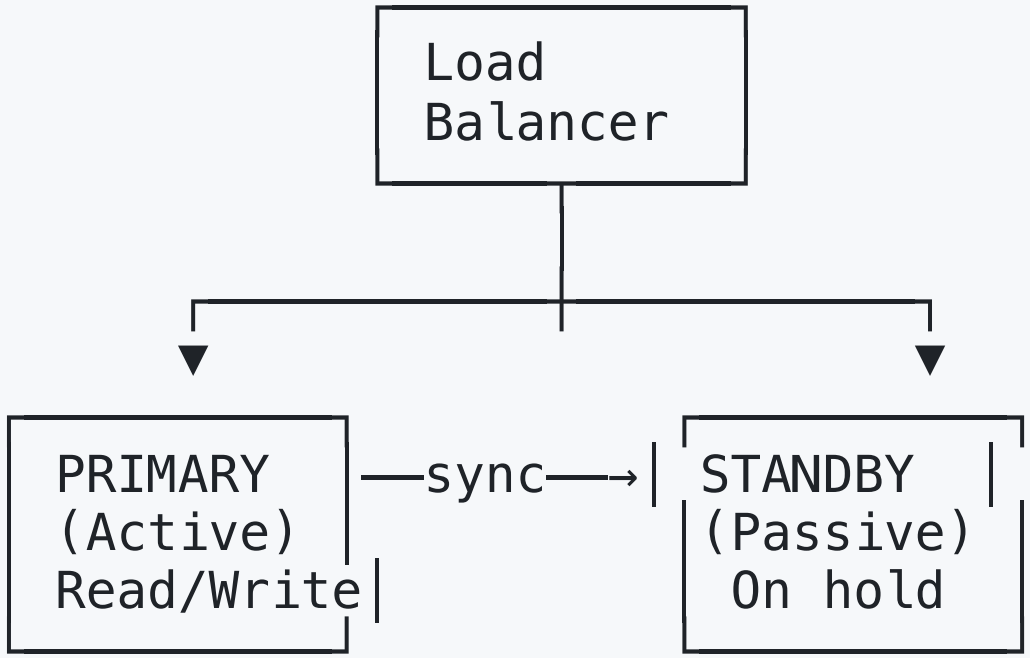
Part 6: High Availability

High Availability (HA) Concepts

Goal: Minimize downtime — keep the system running!

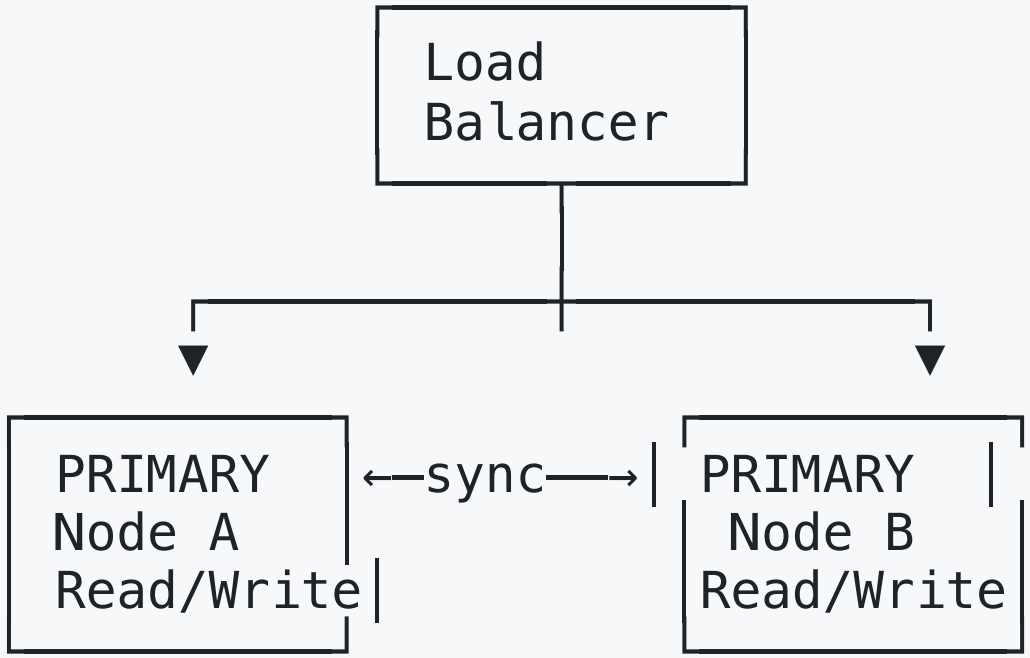
Term	Uptime	Downtime/Year
99%	"Two nines"	3.65 days
99.9%	"Three nines"	8.7 hours
99.99%	"Four nines"	52 minutes
99.999%	"Five nines"	5.2 minutes

HA Architecture: Active-Passive



- **Primary** handles all traffic
- **Standby** takes over on failure (**failover**)
- Can be manual or **automatic** failover

HA Architecture: Active-Active



- **Both** nodes handle read/write traffic
- More complex (conflict resolution needed)
- Better **performance** (load distributed)

Failover Strategies

Strategy	RTO	Description
Manual	Hours	DBA manually promotes replica
Semi-auto	Minutes	Script automates promotion
Automatic	Seconds	Orchestrator detects and failovers

MySQL HA Solutions

Solution	Type	Complexity
MySQL Replication	Active-Passive	Low
MySQL InnoDB Cluster	Group Replication	Medium
MySQL NDB Cluster	Shared-nothing	High
ProxySQL + Orchestrator	Proxy + failover	Medium
Galera Cluster	Sync multi-primary	Medium

Part 7: SQLite Backup Methods

SQLite Backup: Simple File Copy

The simplest backup — SQLite is just a **file**!

```
# Method 1: Copy the database file
cp university.db university_backup_$(date +%F).db

# Method 2: Copy while ensuring no writes
sqlite3 university.db "BEGIN IMMEDIATE; .backup backup.db; COMMIT;"
```

⚠️ **Warning**: Don't copy while writes are happening!

Use the `.backup` command or the Online Backup API.

SQLite .backup Command

```
-- Inside sqlite3 shell:

-- Backup entire database
.backup university_backup.db

-- Backup to a specific path
.backup '/path/to/backups/university_2025-01-15.db'

-- Backup a specific attached database
.backup main '/backups/main_backup.db'
```

The `.backup` command uses SQLite's **Online Backup API** — safe even while the database is in use!

SQLite .dump Command (Logical Backup)

```
.dump -- Dump entire database as SQL

-- Dump to a file
.output university_dump.sql
.dump
.output stdout

-- Dump specific tables
.dump student
.dump student course enrollment
```

Output is SQL text (like mysqldump):

```
BEGIN TRANSACTION;
CREATE TABLE student (...);
INSERT INTO student VALUES (...);
COMMIT;
```

SQLite Restore

```
# Restore from .backup (physical)
cp university_backup.db university.db

# Restore from .dump (logical)
sqlite3 university_restored.db < university_dump.sql

# Restore inside sqlite3 shell
.read university_dump.sql
```

SQLite WAL Mode and Backup

```
-- Check journal mode
PRAGMA journal_mode;

-- Enable WAL for better concurrent access
PRAGMA journal_mode = WAL;
```

WAL (Write-Ahead Logging):

- Allows concurrent readers and one writer
- Backups can run while database is in use
- Better performance for most workloads

MySQL vs SQLite Backup Comparison

Feature	MySQL	SQLite
Logical backup	mysqldump	.dump
Physical backup	XtraBackup / file copy	.backup / file copy
Point-in-time	Binary logs	Journal files (limited)
Replication	Built-in	Not built-in
HA / Failover	InnoDB Cluster	Application level
Complexity	High	Low

Part 8: Practice Exercises

Exercise 1: Backup Strategy Design

Design a backup strategy for an e-commerce database that:

- Has 50GB of data
- Processes 1000 orders/day
- Requires RPO of 1 hour
- Requires RTO of 30 minutes

What tools and schedule would you use?

Exercise 1: Sample Answer

Component	Choice
Full backup	mysqldump weekly (Sunday 2 AM)
Incremental	Binary logs continuously
Replication	Semi-synchronous to hot standby
RPO achieved	~0 (semi-sync replication)
RTO achieved	~5 min (auto-failover to standby)
Offsite	Daily backup to cloud storage

Exercise 2: RPO/RTO Calculation

A company does:

- Full backup every Sunday at midnight
- Incremental backups every 6 hours
- The system crashes on Wednesday at 3:45 PM

Questions:

1. What is the maximum data loss (RPO)?
2. If restore takes 2 hours for full + 30 min per incremental, what is RTO?

Exercise 2: Answers

1. **RPO**: Last incremental was at noon Wednesday. Max data loss = **3 hours 45 minutes**
2. **RTO**:
   - Full restore: 2 hours
   - Incrementals: Sunday midnight → Wed noon = ~13 incremental backups × 30 min = 6.5 hours
   - Total RTO ≈ **8.5 hours**

   Better strategy: Use continuous binary logs + semi-sync replication

Exercise 3: SQLite Backup Practice

```
-- Create a backup of the university database
.backup university_backup.db

-- Create a SQL dump
.output university_dump.sql
.dump
.output stdout

-- Verify the dump
-- .read university_dump.sql  (into a new database)
```

Exercise 4: Replication Quiz

Question	Answer
Default MySQL replication type?	?
Semi-sync waits for how many ACKs?	?
Replicas handle which operations?	?
What do replicas use to apply changes?	?

Exercise 4: Answers

Question	Answer
Default MySQL replication type?	**Asynchronous**
Semi-sync waits for how many ACKs?	**At least 1**
Replicas handle which operations?	**Reads only**
What do replicas use to apply changes?	**Binary logs**

📝 Homework

1. **Theory:**
    - Design a DR (Disaster Recovery) plan for a hospital database with RPO=0, RTO<5 min
    - Compare mysqldump vs XtraBackup for a 500GB database
    - Explain when you would choose Group Replication vs Asynchronous
2. **SQLite Practice:**
    - Create a backup script that:
        - Creates a timestamped .backup
        - Creates a .dump SQL file
        - Verifies the backup by restoring to a temp database

🧠 Key Exam Concepts

Topic	What to Remember
mysqldump	Logical backup → SQL text file
--single-transaction	Consistent InnoDB backup
Binary logs	Record all changes for incremental/PITR
RPO	How much data loss is acceptable
RTO	How much downtime is acceptable
Async replication	Default, fastest, may lose data
Semi-sync	Waits for 1 ACK, better durability
HA	Auto-failover for near-zero downtime
3-2-1 Rule	3 copies, 2 media, 1 offsite
SQLite .backup	Safe hot backup using Online Backup API

Next Week Preview

Week 15: Practical Studies

- Hands-on project presentation
  - Randomly selected groups to present
- Final exam review

Thank You!

Questions?